

Omer Rehan

24K-0760

BSCS-4K DB-Assignment #3.

Date 20
M T W T F S S

Q1.

(1) π Platform.name (σ Difficulty.level = 'Advanced' $\wedge$ year (Enrol.Date) = 2016 (PLATFORM $\bowtie$ ENROLLMENT $\bowtie$ COURSE))

Explanation:

It selects platform names where advance-level courses had enrollments in 2016.

(2) π Instructor.Name (σ Status = 'Completed' $\wedge$ Category = 'Data Science' (INSTRUCTOR $\bowtie$ COURSE ACTIVITY $\bowtie$ ENROLLMENT $\bowtie$ COURSE))

This joins course activities, details to their instructors filtering "Data Science" courses that are "Completed" and shows instructor's name.

(3) π Platform.name (PLATFORM) - π Platform.name (PLATFORM $\bowtie$ INSTRUCTORS)

It use set difference operation to subtract names of platforms that have atleast one registered instructor

(4) π C1.course.Name (σ C1.course.Id = C2.course.Id $\wedge$ E1.platform.Id $\neq$ E2.platform.Id (PE1 (ENROLLMENT) $\bowtie$ PE2 (Enrollment) $\bowtie$ PC1 (COURSE)))

It finds courses whose same course-id appears on multiple platforms.

(5) A Course-Name, Platform-Name, Completion Rate (category = 'AI'
A completion_Rate > 85 (Course A Enrollment &
Platform))

It shows AI courses along with platforms where
completion rate is above 85%.

Question #2.

(1) It shows titles of Books that are unavailable and
have price greater than 1000.

```
SELECT TITLE
```

```
FROM BOOK
```

```
WHERE isAvailable = 0 AND price > 1000;
```

(2) It shows titles of fiction books above 800 or All history Books.

```
SELECT TITLE
```

```
FROM BOOK
```

```
WHERE (genre = 'Fiction' AND price > 800)  
OR genre = 'History';
```

(3) It shows title and price of available science books.

```
SELECT title, price
```

```
FROM BOOK
```

```
WHERE genre = 'Science' AND isAvailable = 1.
```

(4) shows title and 10% of price of available science books

```
SELECT title, price / 10 as discounted-price
FROM Book
Where genre = 'Science' AND isAvailable = 1;
```

(5) It finds publisher ids of publishers who published "AI Books"

```
SELECT P.publisherId
FROM Publishes P
JOIN book B on P.bookId = B.BookId
Where B.title = "AI Basics";
```

(6) It shows staff names and library names where staff works in a library located in same city as their address.

```
SELECT S.name, L.libName
FROM Staff S
JOIN Library L on S.libId = L.libId
Where S.address = L.city;
```

(7) Shows staff ids of employees whose salary is not 60000

```
SELECT staffId
FROM Staff
Where salary <> 60000;
```

(8) it finds publishers IN that are present in Karachi and Islamabad sets.

```
SELECT publisherId
FROM Publisher
where city = 'Karachi'
INTERSECT
SELECT publisherId
FROM Publisher
where city = 'Islamabad';
```

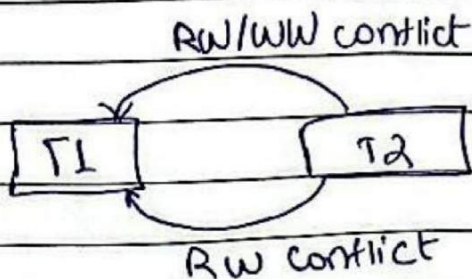
```
(a) SELECT Title
FROM Book
EXCEPT
SELECT Title FROM Book NATURAL JOIN Publisher;
```

Question #3

(a) Non Serial Schedule : $r_1(A), r_2(A), w_1(A), w_2(A)$.

(b) Precedence Graph.

Steps	T1	T2
1	$r_1(A)$	
2		$r_2(A)$
3	$A := A - N$	
4	$w_1(A)$	
5		$A := A + M$
6		$w_2(A)$



(c) Since the precedence graph has cycle $T1 \rightarrow T2$ and $T2 \rightarrow T1$ therefore this schedule is not conflict serializable.

(d)

Option 1:

1. $x_1(A) - T1$

2. $A := A - N - T1$

3. $w_1(A) - T1$

4. $y_2(A) - T2$

5. $A := A + M - T2$

6. $w_2(A) - T2$ ($A = (A - N) + M$)

Option 2:

(vice versa).

(e) Real world impact:

It can cause loss update problem. If two users try to withdraw and deposit money at same time, database overwrites one or the transactions. This only reflects $T2$'s update completely ignoring $T1$'s transaction leading to severe financial data inaccuracy.

Question #4.

Schedule S1:

On X:

$r_1(x)$ before $w_3(x) \rightarrow T1 \rightarrow T3 (RW)$

$r_3(x)$ before $w_1(x) \rightarrow T3 \rightarrow T1 (RW)$

$w_1(x)$ before $w_3(x) \rightarrow T1 \rightarrow T3 (WW)$

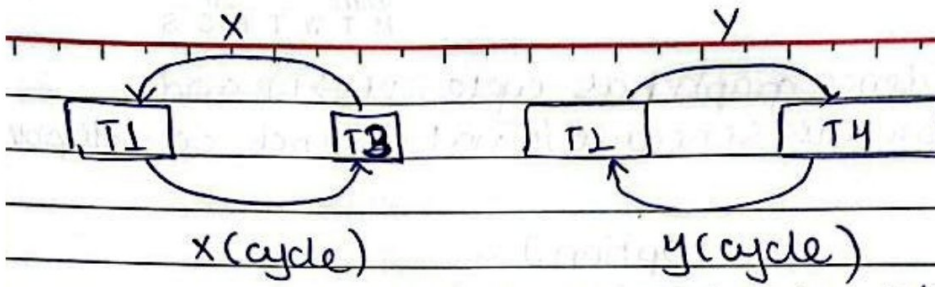
On Y:

$r_2(y)$ before $w_4(y) \rightarrow T2 \rightarrow T4 (RW)$

$r_4(y)$ before $w_2(y) \rightarrow T4 \rightarrow T2 (RW)$

$w_2(y)$ before $w_4(y) \rightarrow T2 \rightarrow T4 (WW)$





Two cycles $\rightarrow$ NOT Serializable $\rightarrow$ NO serial order.

Results:

Since cycles $T1 \rightarrow T3$ and $T2 \rightarrow T4$ exist which means conflict serializability is impossible.

Schedule S_2 : $\rightarrow r_3(x), w_3(x), r_1(x), r_1(y), w_1(x), r_2(y), w_2(y), r_4(y), w_4(y)$
Conflict Pairs: $r_4(y), w_4(y)$.

(1)

On x:

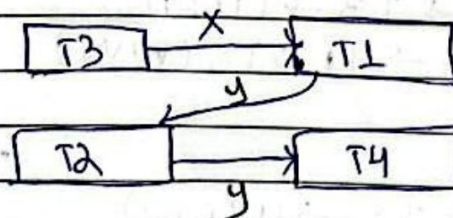
$w_3(x)$ before $r_1(x) \rightarrow T3 \rightarrow T1$ (WR)
 $w_3(x)$ before $w_1(x) \rightarrow T3 \rightarrow T1$ (WW)

On y:

$r_1(y)$ before $w_2(y) \rightarrow T1 \rightarrow T2$ (RW)
 $r_2(y)$ before $w_4(y) \rightarrow T2 \rightarrow T4$ (RW)
 $w_2(y)$ before $w_4(y) \rightarrow T2 \rightarrow T4$ (WW)

no cycles $\rightarrow S_2$ is conflict serializable.

(2) Precedence Graph.



Equivalent Serial Order:

$T3, T1, T2, T4$
($T3 \rightarrow T1 \rightarrow T2 \rightarrow T4$).

No cycles $\rightarrow$ serializable

Question 5:

Initial Balance: 5000

Problem: Lost update

Interleaved Execution:

Step	T1	T2	A in DB
1	reads $A = 5000$		5000
2		reads $A = 5000$	5000
3	$A = 5000 - 1000 = 4000$		5000
		$A = 5000 + 2000 = 7000$	5000
	$A = 4000$		4000
		$A = 7000$	7000 ← Lost update.

Full Balance: \$7000

T1's write of 4000 is overwritten by T2's write of 7000 due to which subtraction of 1000 is lost.

Correct Balance:

If T1 first, then T2: $5000 - 1000 = 4000 \rightarrow 4000 + 2000 = \6000

If T2 first, then T1: $5000 + 2000 = 7000 - 1000 = \6000

Correct Result:

serial order gives \$6000. The result \$7000 is wrong



Question #6:

(a) Timestamp Ordering Protocol:

Explanation:

The timestamp Ordering protocol assigns a unique timestamp (ticket number) ^{at service counter} to every transaction when it arrives. There is a strict rule that operation must proceed in timestamp order.

Problems in Basic Systems

Example:

If T1 gets timestamp 1

T2 gets timestamp 2

So T1 should execute before T2

Each data item has 2 values:

Read Timestamp: latest transaction that read item

Write Timestamp: latest transaction that wrote item

If a transaction tries to read/write in way that breaks timestamp order it is rejected and restarted.

Problems with Basic Timestamp Ordering:

1- Starvation: A transaction that keeps getting aborted due to new transactions being ahead may never fully complete. It receives a new timestamp each restart and gets behind leading to starvation.



2. Cascading Aborts:

If T_i reads value written by T_j and T_j is later aborted, T_i must also be aborted and any transaction that reads T_i 's writes must be aborted too which creates a cascade.

Strict Timestamp Ordering:

In

Strict timestamp ordering a transaction is not allowed to read or write data item X until all transactions with earlier timestamps that wrote X have either committed or aborted. This prevents dirty reads and cascading rollbacks.

Example:

If T_1 (older) writes to Account A, and T_2 (younger) wants to read account A, T_2 is not allowed to read it until T_1 officially commits or aborts. This prevents T_2 from reading a messy "dirty" value, avoiding cascading aborts if T_1 fails later.

Question 06(b): Deadlock.

Step	T_1	T_2
1-	Acquires lock on A	
2		Acquires lock on B
3	Request lock on B $\rightarrow$ WAIT (held by T_2)	Request lock on A $\rightarrow$ WAIT (held by T_1)

So now T_1 waits for T_2 and T_2 waits for T_1 a circular wait which leads deadlock.

Prevention methods:

1- wait-Die / wound-wait

wait/Die: If T_1 requests a resource held by T_2 , T_1 is allowed to wait only if it is older than T_2 . If T_1 is younger, it "dies" (aborts and restarts).

wound wait: An older transactions forces rollback of a younger one. T_1 (older) wounds T_2 , forcing T_2 to restart.

2- Deadlock Detection: In this system maintains a wait for graph. Periodically checks for cycles. When cycle is found ($T_1 \rightarrow T_2, T_2 \rightarrow T_1$) one drops select victim transaction and aborts it and other transactions then proceed.

